Docker

Résumé du chapitre : gérer les conteneurs, construire et optimiser des images

1. Introduction

Docker est une plateforme pour construire des images et exécuter des conteneurs. Ses commandes sont très proches de celles de Podman.

Opération	Docker	Podman
Lancer un conteneur	docker run	podman run
Lister les conteneurs	docker ps	podman ps
Arrêter un conteneur	docker stop	podman stop
Construire une image	docker build	podman build
Copier des fichiers	docker cp	podman cp
Réseaux / Volumes	docker network / volume	podman network / volume

À retenir : différence d'architecture : Docker Engine utilise un service central (dockerd), alors que Podman peut fonctionner sans démon permanent.

2. Construire une image avec un Dockerfile

- Fichier par défaut : **Dockerfile** (Docker) ou **Containerfile** (Podman). Les deux outils acceptent les deux noms.
- Commande : `docker build -t myapp .`
- Instructions principales : FROM, RUN, COPY, CMD, ENTRYPOINT...

CMD et ENTRYPOINT

Les deux définissent ce qui s'exécute au démarrage du conteneur :

- **CMD** : commande ou arguments par défaut, faciles à remplacer au lancement.
- **ENTRYPOINT** : le programme principal du conteneur ; les arguments de `docker run` lui sont transmis.

Situation	Comportement
CMD seul	La commande par défaut peut être remplacée
ENTRYPOINT seul	Programme fixe ; on peut ajouter des arguments
ENTRYPOINT + CMD	Programme fixe avec arguments par défaut modifiables

```
FROM alpine:3.22
ENTRYPOINT ["echo"]
CMD ["Hello Docker"]

# docker run --rm echo-tool       -> Hello Docker
# docker run --rm echo-tool "Hello" -> Hello
```

À retenir : l'association ENTRYPOINT + CMD crée une image qui se comporte comme un outil exécutable avec un argument par défaut.

3. Les layers d'une image

Une image est faite de plusieurs couches (*layers*) qui représentent les modifications du système de fichiers pendant le build.

- RUN, COPY, ADD créent normalement des couches de fichiers.
- CMD, ENV, WORKDIR modifient la configuration sans forcément créer de couche.

Avantages

- **Réutilisation** : des images peuvent partager les mêmes couches.
- **Économie d'espace** : pas de duplication inutile.
- **Rapidité** : les étapes déjà construites sont réutilisées.
- **Distribution** : seules les couches manquantes sont téléchargées.

Stockage

Élément	Rôle
containerd	Gère notamment les images et les conteneurs
Blobs	Contiennent les données des layers (content store de containerd)
Snapshots	Combinent les layers pour former le système de fichiers du conteneur
OverlayFS	Superpose les couches sous Linux et ajoute une couche inscriptible

```
docker info                              # infos de stockage
docker image inspect custom-nginx:1.0     # détails d'une image
docker image history custom-nginx:1.0     # historique des couches
```

4. Le cache de build

- Docker lit le Dockerfile dans l'ordre et réutilise les étapes inchangées (indication **CACHED** avec BuildKit).
- `--no-cache` force la réexécution de toutes les étapes : utile pour diagnostiquer ou éviter un résultat d'installation en cache.
- **BuildKit** est le moteur de construction utilisé par Docker.

Optimiser le cache : l'ordre compte

Placer ce qui change rarement avant ce qui change souvent.

```
FROM node:22-alpine
WORKDIR /app
COPY package.json package-lock.json ./   # dépendances d'abord
RUN npm ci                                # reste en cache si inchangé
COPY . .                                  # code source ensuite
CMD ["npm", "start"]
```

Un fichier `.dockerignore` (ex. .git, .env, *.log) réduit le contexte de build et évite d'inclure des fichiers inutiles ou sensibles.

5. Le build multi-stage

Plusieurs étapes dans un même Dockerfile : on prépare l'application dans une première étape, puis l'image finale ne garde que ce qui est nécessaire à l'exécution.

```
# Étape 1 : dépendances
FROM python:3.12 AS builder
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir --prefix=/install -r requirements.txt
```

```
# Étape 2 : image finale
FROM python:3.12-slim
WORKDIR /app
COPY --from=builder /install /usr/local
COPY app.py .
CMD ["python", "app.py"]
```

À retenir : COPY --from=builder copie uniquement les fichiers nécessaires depuis une étape précédente vers l'image finale.

6. Optimiser une image Docker : 6 bonnes pratiques

#	Bonne pratique	Détail
1	Image de base légère	ex. python:3.12-slim plutôt qu'une image complète
2	Fichier .dockerignore	exclure .git, .venv, __pycache__, fichiers temporaires, secrets
3	Ordre des instructions	dépendances d'abord, code source ensuite
4	Cache de build	laisser Docker réutiliser les étapes inchangées
5	Multi-stage build	ne garder que le nécessaire à l'exécution
6	Limiter dépendances et privilèges	paquets utiles seulement, utilisateur non privilégié

À retenir : l'objectif est une image aussi légère que possible, sans retirer le nécessaire, tout en restant fonctionnelle et maintenable.

7. Questions de compréhension : corrigé

1. Différence entre une image et un conteneur ?

Une **image** est un modèle en lecture seule, composé de couches, qui contient l'application et ses dépendances. Un **conteneur** est une instance en cours d'exécution (ou arrêtée) d'une image : il ajoute une couche inscriptible et un processus isolé. On peut créer plusieurs conteneurs à partir d'une même image.

2. Rôle de FROM ?

`FROM` définit l'**image de base** sur laquelle le build commence (ex. `FROM alpine:3.22`). C'est la première instruction d'un Dockerfile (ou de chaque étape d'un multi-stage), et elle peut recevoir un alias avec AS.

3. Différence entre RUN et CMD ?

`RUN` s'exécute **pendant le build** : il lance une commande (installation de paquets...) et crée une couche dans l'image. `CMD` s'exécute **au démarrage du conteneur** : il définit la commande par défaut et ne crée pas de couche de fichiers.

4. Rôle de ENTRYPOINT ?

`ENTRYPOINT` définit le **programme principal** du conteneur, exécuté à chaque démarrage. Les arguments donnés à `docker run` lui sont transmis plutôt que de le remplacer (sauf avec `--entrypoint`).

5. Différence entre CMD et ENTRYPOINT ?

CMD fournit une commande ou des arguments **par défaut**, entièrement remplaçables en ajoutant une commande à `docker run`. **ENTRYPOINT** fixe le **programme** à exécuter ; les arguments du `docker run` s'ajoutent à lui au lieu de le remplacer.

6. Pourquoi associer ENTRYPOINT et CMD ?

Pour obtenir une image qui se comporte comme un **outil exécutable** : ENTRYPOINT fixe le programme, CMD donne un argument par défaut que l'utilisateur peut modifier. Exemple : `ENTRYPOINT ["echo"]` + `CMD ["Hello Docker"]` affiche « Hello Docker » par défaut, ou le texte fourni au lancement.

7. Qu'est-ce qu'un layer ?

Un **layer** (couche) est un ensemble de modifications du système de fichiers (fichiers ajoutés, modifiés ou supprimés) produit par une instruction du Dockerfile comme RUN, COPY ou ADD. L'image est l'empilement de ces couches en lecture seule.

8. Avantages des layers ?

Réutilisation (couches partagées entre images), **économie d'espace** (pas de duplication), **rapidité** des builds (étapes déjà construites réutilisées) et **distribution** efficace (seules les couches manquantes sont téléchargées).

9. Rôle du cache de build ?

Il permet à Docker de **réutiliser les étapes inchangées** d'un build précédent (indiquées *CACHED* avec BuildKit) au lieu de les réexécuter. Les builds sont ainsi beaucoup plus rapides. Dès qu'une étape change, elle et toutes les suivantes sont reconstruites.

10. Différence entre --no-cache et --pull ?

`--no-cache` force la **réexécution de toutes les étapes** du Dockerfile sans utiliser le cache. `--pull` force Docker à **télécharger la version la plus récente de l'image de base** (FROM), même si une copie locale existe. Les deux peuvent être combinés pour un build totalement « propre ».

11. Comment organiser un Dockerfile pour le cache ?

Placer ce qui change **rarement** avant ce qui change **souvent**. Par exemple, copier d'abord les fichiers de dépendances (`package.json`, `requirements.txt`), installer les dépendances, puis copier le code source. Ainsi, une modification du code ne relance pas l'installation des dépendances.

12. À quoi sert .dockerignore ?

Il liste les fichiers et dossiers (.git, .env, *.log, .venv, __pycache__...) **exclus du contexte de build**. Cela réduit la taille du contexte envoyé à Docker, accélère le build, améliore l'usage du cache et évite d'inclure des fichiers inutiles ou sensibles (secrets) dans l'image.

13. Comment examiner l'historique d'une image ?

Avec `docker image history <image>` (ex. `docker image history custom-nginx:1.0`), qui affiche les couches, les instructions qui les ont créées et leur taille. Pour plus de détails (config, couches, métadonnées) : `docker image inspect <image>`.

14. Commande pour consulter les logs d'un conteneur ?

`docker logs <conteneur>`. L'option `-f` suit les logs en temps réel, `--tail N` n'affiche que les N dernières lignes. Avec Podman : `podman logs`.

15. Pourquoi un stockage persistant pour une base de données ?

Les données écrites dans la couche inscriptible d'un conteneur sont **perdues quand le conteneur est supprimé**. Pour une base de données, il faut un **volume** (ou bind mount) afin que les données survivent à l'arrêt, à la suppression ou au remplacement du conteneur (ex. mise à jour de l'image), et puissent être sauvegardées ou réutilisées par un autre conteneur.